Git Command Cheatsheet

C171 | v3.0

Setup, snapshot, inspect and investigate

Inspect	Review	Stage	Verify	Commit	Publish
`status`	`diff`	`add`	`diff --staged`	`commit`	`push`

Setup and snapshot

Install, identify yourself, start a repository and record snapshots.

`git --version`
Verify that Git is installed and available.

`git config --global user.name "Your Name"`
Set the author name used in new commits.

`git config --global user.email "you@example.com"`
Set the author email used in new commits.

`git config --global init.defaultBranch main`
Make main the initial branch of new repositories.

`git init`
Create a new local Git repository in the current folder.

`git clone <url>`
Copy a remote repository and configure origin.

`git status -sb`
Show the branch plus concise working-tree and staging state.

`git diff`
Inspect unstaged changes before selecting them.

`git add <file>`
Stage a named file for the next commit.

`git add -p`
Select individual change hunks to stage.

`git diff --staged`
Review exactly what the next commit will contain.

`git commit -m "Message"`
Create a local snapshot with an outcome-based message.

`git commit --amend`
Rewrite the most recent commit; only before it is shared.

`git rm --cached <file>`
Stop tracking a file that .gitignore cannot hide retroactively.

Inspect and investigate

Read history, objects and line-level authorship.

`git log --oneline --graph --all --decorate`
Read compact history, branches and pointers.

`git log --since="2 weeks" --author=<name>`
Filter history by time window and author.

`git log -- <path>`
Show only the commits that touched one path.

`git log -S"<text>"`
Find commits that added or removed a string.

`git show HEAD`
Inspect the latest commit and its patch.

`git cat-file -p <sha>`
Print a raw blob, tree, commit or tag object.

`git blame <file>`
Show which commit and author last changed each line.

`git diff <a>..<b>`
Compare any two commits, branches or tags.

Sources: git-scm.com/docs | github.com/tertiarycourses/githubtutorial | w3schools.com/git

Git Command Cheatsheet

C171 | v3.0

Branch, sync, tag, stash and rescue

Branch	Commit	Fetch	Integrate	Tag	Rescue
`switch -c`	`commit`	`fetch`	`merge/rebase`	`tag -a`	`reflog`

Branch, merge and rebase

Isolate work and integrate it with the right strategy.

`git branch`
List local branches and identify the current branch.

`git switch -c <branch>`
Create and switch to a focused branch.

`git switch main`
Return to the main branch.

`git switch -`
Jump back to the previously checked-out branch.

`git merge <branch>`
Integrate a completed branch into the current branch.

`git merge --no-ff <branch>`
Force a merge commit so the branch remains visible.

`git rebase main`
Replay local commits on top of main; never on shared history.

`git rebase --abort`
Cancel an in-progress rebase and restore the prior state.

`git cherry-pick <commit>`
Copy one commit onto the current branch.

`git branch -d <branch>`
Delete a branch only after Git confirms it is merged.

Sync and remote

Move history between the local repository and GitHub.

`git remote -v`
Inspect remote fetch and push endpoints.

`git remote add upstream <url>`
Register the source repository of a fork.

`git fetch origin`
Update remote-tracking references without changing working files.

`git branch -vv`
Inspect upstream tracking plus ahead/behind information.

`git log HEAD..origin/main --oneline`
List incoming commits before integrating them.

`git pull --ff-only origin main`
Fetch and fast-forward without creating an unexpected merge commit.

`git push -u origin <branch>`
Publish a branch and establish its upstream.

`git push origin --tags`
Publish annotated tags so releases are visible on GitHub.

Tag, stash and release

Mark releases and park work in progress.

`git tag`
List the tags that already exist in the repository.

`git tag -a v1.0.0 -m "Release 1.0.0"`
Create an annotated tag with author, date and message.

`git tag v1.0.0-light`
Create a lightweight tag: a bare pointer with no metadata.

`git show v1.0.0`
Inspect the tag object and the commit it marks.

`git stash push -m "wip"`
Park uncommitted work and return to a clean tree.

`git stash list`
List parked stash entries in order.

`git stash pop`
Reapply the newest stash and drop it from the list.

`git stash apply stash@{1}`
Reapply an older stash while keeping it stored.

Undo and rescue

Choose the safe undo for the state and sharing status.

`git restore <file>`
Discard an unstaged local change to a file.

`git restore --staged <file>`
Unstage a file while keeping its working-tree edits.

`git reset --soft <commit>`
Move HEAD only; keep the index and working tree.

`git reset --mixed <commit>`
Move HEAD and reset the index; keep the working tree.

`git reset --hard <commit>`
Move HEAD, index and working tree; discards work.

`git revert <commit>`
Create a new commit that safely reverses shared history.

`git reflog`
List where HEAD has been, so lost commits can be recovered.

`git switch -c rescue <sha-from-reflog>`
Recover a detached or reset commit onto a new branch.

Sources: git-scm.com/docs | github.com/tertiarycourses/githubtutorial | w3schools.com/git